Assignment 03: Deploy a Static Website to Multiple Servers Using Ansible

DevOps Micro Internship (DMI) Cohort 3 with Agentic AI

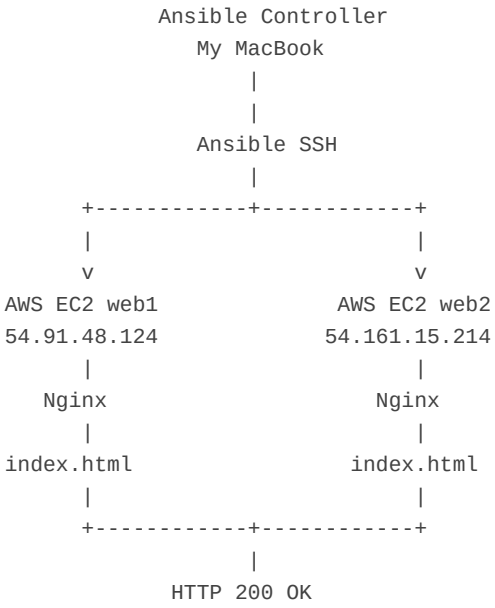
Student Details

Full Name:	Kafayat Olaide Aziz
Cloud Platform:	AWS
Server 1:	http://54.91.48.124
Server 2:	http://54.161.15.214

Project Overview

This project demonstrates how Ansible can be used to automate the deployment of a static website to multiple Ubuntu servers. Using two AWS EC2 instances as web servers, I created an Ansible inventory containing both servers and used a multi-play Ansible playbook to: install Nginx on both servers, start and enable the Nginx service, deploy a personalized static website to both servers, reload Nginx after deployment, verify that both websites return HTTP 200, and test idempotency by running the playbook again.

Architecture



Project Structure

```
static-web/
■■■ inventory.ini
■■■ site.yml
■■■ README.md
■■■ files/
    ■■■ index.html
```

- **inventory.ini** contains the details of the two managed servers.
- **site.yml** contains the multi-play Ansible playbook.
- **files/index.html** contains the static website that is deployed.
- **README.md** documents the project and what was learned.

Ansible Inventory

The inventory defines the two Ubuntu servers that Ansible manages. The servers are grouped under [web], which allows the playbook to target both servers together.

```
[web]
web1 ansible_host=54.91.48.124
web2 ansible_host=54.161.15.214

[web:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/terraform-aws-vm-key
```

Multi-Play Ansible Playbook

The **site.yml** playbook contains three separate plays, each with a specific responsibility:

Play 1: Install and Configure Nginx

The first play targets the web group. It updates the APT package cache, installs Nginx, starts Nginx, and enables Nginx so it starts automatically.

Play 2: Deploy the Website

The second play copies **files/index.html** from the Ansible controller to **/var/www/html/index.html** on each server. The file is owned by www-data and uses permissions of 0644. The play also uses a handler to reload Nginx when the website file changes.

Play 3: Verify the Website

The third play runs on the Ansible controller. It uses the Ansible uri module to send HTTP requests to both servers and checks that they return HTTP 200. This confirms that the websites are accessible.

Deployment Process

1. Configure the Inventory

The two AWS EC2 instances were added to inventory.ini as web1 and web2. Verified the inventory using: **ansible-inventory -i inventory.ini --graph**

2. Test Server Connectivity

Tested the connection to both servers using: **ansible web -i inventory.ini -m ping**
Both servers returned SUCCESS and "ping": "pong". This confirmed that Ansible could communicate with both servers.

3. Install and Configure Nginx

The first play of the playbook installed Nginx and ensured that the service was running and enabled.

4. Deploy the Website

The personalized index.html file was copied to both servers using the Ansible copy module. The website was personalized with the full name: **Kafayat Olaide Aziz**

5. Verify the Deployment

The third play used the uri module to check both servers. A successful HTTP 200 response confirmed that the website was being served correctly.

Idempotency

The playbook was run a second time to test idempotency—the principle that Ansible should not repeatedly make changes when the servers are already in the desired state. During the first run, Nginx was installed and the website was deployed. During the second run, Ansible checked the existing configuration and files; tasks already in the desired state did not require changes. This demonstrates one of the important benefits of configuration management with Ansible.

Issues Encountered

SSH Connection Timeout: The AWS security group was allowing SSH access from an old IP address (102.88.167.200/32), but the current public IP address was 102.216.11.5. Because the current IP was not allowed, the SSH connection timed out.

Resolution: Checked the AWS security group and updated the SSH rule to allow the current IP address. An SSH host key verification issue was resolved by disabling host key checking for the Ansible connectivity test. After these changes, Ansible successfully connected to both servers and returned SUCCESS and pong.

What I Learned

This assignment helped understand how Ansible can automate server management and deployment. Key learnings include:

- Creating an Ansible inventory
- Managing multiple servers using an Ansible group
- Testing connectivity using the ping module
- Creating a multi-play Ansible playbook
- Installing packages using Ansible
- Managing services using Ansible
- Deploying files using the copy module
- Using handlers to reload services after changes
- Using the uri module to test HTTP responses
- Understanding Ansible idempotency
- Troubleshooting SSH and AWS security group connectivity issues

Why Multi-Play Automation Is Useful

Separating the playbook into installation, deployment, and verification makes automation easier to understand and troubleshoot. Each play has a clear responsibility:

Play 1: Install and configure Nginx
Play 2: Deploy the website
Play 3: Verify the website

If a problem occurs, it's easier to identify which stage of the deployment caused it instead of troubleshooting the entire process at once.

Why Use the Ansible Copy Module?

The copy module allows the Ansible controller to send the exact website file to each managed server. This ensures that both servers receive the same version of the website. It also means that Git does not need to be configured separately on every managed server just to deploy the static file.

Website URLs

Server 1: http://54.91.48.124
Server 2: http://54.161.15.214

Both servers were tested to confirm that the website was accessible.

Assignment Questions & Answers

Q1: What issue did you face while completing this assignment, and how did you fix it?

I initially had an SSH connection timeout because the AWS security group only allowed SSH connections from an old IP address. I checked my current public IP address and updated the security group to allow SSH from my current IP. I then resolved an SSH host key verification issue so that Ansible could successfully connect to both servers.

Q2: What did you learn from this assignment?

I learned how to use Ansible to manage multiple Ubuntu servers from one controller. I learned how to create an inventory, test connectivity, install Nginx, deploy files, verify HTTP responses, and test idempotency.

Q3: Why is it useful to split installation, deployment, and verification into separate plays?

It makes the playbook easier to understand, maintain, and troubleshoot. Each play has one main responsibility, so it is easier to identify where a problem occurs.

Q4: What is one benefit of using the Ansible copy module instead of cloning the website directly from Git on every managed server?

The copy module allows the controller to deploy the exact tested file to every server. This ensures that both servers receive the same website version without requiring Git to be configured on every server.

Q5: What does idempotency mean in this assignment?

Idempotency means that running the playbook again should not make unnecessary changes when the servers are already correctly configured.

Q6: What does the Ansible uri module verify in Play 3?

The uri module sends an HTTP request to each server and checks the response status. In this assignment, it verifies that both websites return HTTP status 200.

Conclusion

This assignment demonstrated how Ansible can be used to automate the deployment of a static website across multiple servers. Instead of manually installing Nginx and copying the website to each server, I used one Ansible playbook to manage both servers consistently. The project also gave practical experience troubleshooting SSH connectivity, working with AWS security groups, deploying applications with Ansible, and verifying that an automated deployment worked correctly.

This project is part of the **DevOps Micro Internship (DMI) Cohort 3 — Agentic AI Track**, focused on practical DevOps skills, automation, systems thinking, and career readiness.